PET RESCUE, REHABILITATION AND SHELTER MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

KAAVIYA SRI PS

(240701222)

in partial fulfilment of the End Semester Examination for the course

CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

November, 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "Pet Rescue, Rehabilitation and Shelter Management System" is the bonafide work of KAAVIYA SRI PS (240701222), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. MANORANJINI J
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Ms. ASWANA LAL
COURSE FACULTY
Assistant Professor
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, KAAVIYA SRI PS (240701222), hereby declare that the project report titled “Pet Rescue, Rehabilitation and Shelter Management System”, submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai
Date:

SIGNATURE OF THE STUDENT
KAAVIYA SRI PS
240701222

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Thiru. S. Meganathan, Chairperson Dr. (Mrs.) Thangam Meganathan and Vice-Chairman Mr. Abhay Meganathan S for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S. N. Murugesan, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. Manoranjini J, Head of the Department, Department of Computer Science and Engineering, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Mr. Bhuvaneswaran Balasubramanian, Assistant Professor, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform, its trial edition and its learning resources, and I thank my family and friends for their unwavering support.

KAAVIYA SRI PS

ABSTRACT

Animal rescue organisations run on tight resources and fast decisions, yet most of their records still live in notebooks, messaging groups and individual spreadsheets. A rescue is described verbally, injuries are not recorded in a structured way, treatment dates are remembered rather than scheduled, foster homes are arranged by phone and nobody can say with certainty how many cages are free. The Pet Rescue, Rehabilitation and Shelter Management System is a low-code application built on Zoho Creator that brings these activities into one secure system. Zoho Creator was chosen because its forms, reports, Deluge scripting, workflows, role-based permissions and mobile app are available on one hosted platform, which allowed a single student developer to build and deploy a complete application within a semester. The application is organised in three modules. The rescue module records each rescue with the animal's condition on arrival and any injuries, and alerts the veterinarian when a critical animal is admitted or a severe injury is logged. The rehabilitation module records ongoing treatment with a next-due date, e-mails the veterinarian a daily digest of treatments due, registers foster carers with their capacity and accepted species, and validates each foster placement before it is saved. The shelter module maintains the list of cages, assigns animals to cages, refuses assignments to full or unavailable cages and recounts occupancy automatically through reusable Deluge functions, while a scheduled workflow warns the administrator when the shelter is close to capacity. An approval process controls the release of animals, an Integration field and a connection link foster carers and volunteers with Zoho CRM, and role-based permissions with record-level criteria ensure that each user, including external foster carers, sees only the forms, fields and records they need. Dashboards show occupancy, animals by status and treatments due. The application removes manual record keeping, prevents over-filled cages and missed treatments, and gives the administration a live view of the shelter.

Keywords: Low-Code Development; Zoho Creator; Deluge Scripting; Animal Rescue Management; Shelter Capacity Management; Role-Based Access Control

PROJECT AT A GLANCE

Attribute	Details
Project Title	Pet Rescue, Rehabilitation and Shelter Management System
Application Domain	Animal welfare – pet rescue, rehabilitation and shelter administration
Student Name & Register No.	Kaaviya Sri PS – 240701222
Zoho Creator Edition	Zoho Creator 15-day free trial (premium features enabled)
Application Name (Link Name)	PetsCop (petscop)
Published / Live URL	https://creatorapp.zoho.in/kaaviyasri.ps.2024.cse_raja590/petscop
Demo Credentials (per role)	kaaviyasri.ps.2024.cse@rajalakshmi.edu.in pskaaviya123@gmail.com
Number of Forms / Subforms	7 / 0
Number of Reports (by type)	List: 8 Kanban: 1 Calendar: 1 Pivot/Chart: 1 Others: 0
Number of Pages / Dashboards	2 (Shelter Dashboard, My Foster Dashboard)
Number of Workflows	Form: 12 Schedule: 2 Approval: 1 Blueprint: 0 Batch: 0
Deluge Functions / Custom Actions	3 functions (updateCageOccupancy, refreshFosterAvailability, closeCageAssignments)
Integrations	Zoho CRM (contacts; Integration field), Zoho Creator e-mail (sendmail), Creator REST API v2.1 (tested from Postman)
Roles & Profiles Defined	Shelter Administrator, Veterinarian, Rescue Coordinator, Caretaker, Foster Carer, Volunteer
Environments Used	Development → Stage → Production
Date of Production Deployment	31st October 2026 Version 1.2

TABLE OF CONTENTS

ABSTRACT i
PROJECT AT A GLANCE v
LIST OF TABLES vii
LIST OF FIGURES viii
LIST OF ABBREVIATIONS x
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 3
1.6 Organisation of the Report 4
LITERATURE SURVEY AND SYSTEM STUDY 5
2.1 Study of the Existing System 5
2.2 Limitations of the Existing System 6
2.3 Low-Code versus Traditional Development 6
2.4 Literature Survey 7
2.5 Emerging Trends in Low-Code Development 8
2.6 Proposed System 8
REQUIREMENT ANALYSIS 9
3.1 Stakeholders and User Roles 9
3.2 Functional Requirements 10
3.3 Non-Functional Requirements 11
3.4 Hardware and Software Requirements 12
3.5 Use Case Model 13
SYSTEM DESIGN 14
4.1 System Architecture 14
4.2 Data Model 15
4.3 Form Design 16
4.4 Process and Workflow Design 17
4.5 User Interface and Navigation Design 18
4.6 Access Control Design 19
IMPLEMENTATION 20
5.1 Application Setup 20
5.2 Forms and Data Capture 21
5.3 Reports 21
5.4 Pages and Dashboards 22
5.5 Deluge Scripting 23

5.6 Workflows and Automation 24
5.7 Integrations 24
5.8 User Management and Role-Based Access 25
5.9 AI and Analytics Features 25
5.10 Pro-Code Extension 26
5.11 Mobile and Portal Access 27
TESTING AND DEBUGGING 28
6.1 Testing Strategy 28
6.2 Test Cases 28
6.3 Debugging Deluge Scripts 29
6.4 Defect Log 29
6.5 User Acceptance Testing 30
DEPLOYMENT, SECURITY AND GOVERNANCE 31
7.1 Environments and Deployment Strategy 31
7.2 Performance Optimisation 32
7.3 Security Features 32
7.4 Governance and Administration 33
RESULTS AND DISCUSSION 34
8.1 Application Output 34
8.2 Achievement of Objectives 34
8.3 Comparison with the Existing System 35
8.4 Limitations of the Low-Code Approach 36
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 36
CONCLUSION AND FUTURE ENHANCEMENTS 37
9.1 Conclusion 37
9.2 Future Enhancements 38
9.3 Learning Outcomes and Career Relevance 38
REFERENCES 39
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 40
APPENDIX B – COURSE OUTCOME MAPPING 42
APPENDIX C – COMPLETE DELUGE SOURCE CODE 43
APPENDIX D – USER MANUAL 49
D.1 Accessing the Application 49
D.2 Instructions for Staff (Administrator, Veterinarian, Coordinator, Caretaker) 49
D.3 Instructions for Foster Carers and Volunteers 50
APPENDIX E – EVALUATION RUBRIC 51
APPENDIX F – VIVA-VOCE RECORD SHEET 53

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application 6
Table 2.2: Summary of Literature Survey 7
Table 3.1: Stakeholders and User Roles 10
Table 3.2: Functional Requirements 10
Table 3.3: Non-Functional Requirements 12
Table 3.4: Hardware and Software Requirements 12
Table 3.5: Use Case Description – Assign an Animal to a Cage 13
Table 4.1: Entities and Relationships 15
Table 4.2: Field Specification – Cage Master Form 16
Table 4.3: Field Specification – Rescue Report Form 17
Table 4.4: Field Specification – Injury Record Form 17
Table 4.5: Field Specification – Treatment Log Form 18
Table 4.6: Field Specification – Foster Master Form 18
Table 4.7: Field Specification – Foster Placement Form 20
Table 4.8: Field Specification – Holding Assignment Form 23
Table 4.9: Role–Permission Matrix 25
Table 5.1: Reports Implemented 28
Table 5.2: Deluge Scripts Implemented 30
Table 5.3: Workflows Implemented 32
Table 5.4: Integrations Implemented 33
Table 6.1: Test Cases 33
Table 6.2: Defect Log 33
Table 6.3: User Acceptance Feedback 33
Table 7.1: Deployment Checklist 33
Table 7.2: Performance Optimisation Measures 33
Table 7.3: Security Controls Implemented 34
Table 8.1: Mapping of Objectives to Outcomes 35
Table 8.2: Existing System versus Proposed Application 36
Table A.1: Feature Coverage Checklist 36
Table B.1: Mapping of Course Outcomes to Report Chapters 38
Table E.1: Evaluation Rubric for the Individual Project 41
Table E.2: Performance Level Descriptors 44
Table F.1: Viva-Voce Questions and Responses 51
Table F.2: Marks Summary 53

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of Manual Rescue and Shelter Record Keeping 5
Figure 3.1: Use Case Diagram of the Pet Rescue System 13
Figure 4.1: System Architecture of the Pet Rescue System 14
Figure 4.2: Entity–Relationship Diagram of the Pet Rescue System 15
Figure 4.3: Process Flow for Cage Assignment Validation 19
Figure 4.4: Navigation Structure of the Pet Rescue System 20
Figure 5.1: Application Home – Navigation Sidebar with the Rescue, Rehabilitation and Shelter Sections 21
Figure 5.2: Rescue Report Form in the Form Builder 22
Figure 5.3: Holding Assignment Form in Live Mode – Cage Full Alert 23
Figure 5.4: Animal Status Board (Kanban) and Treatment Calendar Reports 24
Figure 5.5: Shelter Dashboard 25
Figure 5.6: Veterinarian and Foster Carer Permission Sets 30
Figure 6.1: Running refreshFosterAvailability with Execute in the Deluge Function Editor 33
Figure 7.1: Environments – Publish to Stage and Production 34
Figure 7.2: Schedule Execution Logs of the Application (Operations > Logs) 35
Figure 8.1: Scenario Step 1 – Critical Rescue Recorded and Vet Alerted 36
Figure 8.2: Scenario Step 2 – Cage Assignment and Full-Cage Alert 37
Figure 8.3: Scenario Step 3 – Foster Placement and the Foster Carer’s View 37

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CO	Course Outcome
CRM	Customer Relationship Management
DS	Deluge Script
ER	Entity–Relationship
FR	Functional Requirement
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
LCDP	Low-Code Development Platform
NFR	Non-Functional Requirement
OAuth	Open Authorization
RBAC	Role-Based Access Control
REST	Representational State Transfer
UAT	User Acceptance Testing
UI	User Interface
UC	Use Case
Zia	Zoho’s AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

Animal rescue organisations take in injured, abandoned and abused animals, nurse them back to health and either release them or place them in temporary foster homes. The work is continuous and unpredictable: a rescue call can arrive at any hour, the animal may need emergency treatment, and the shelter has only a limited number of cages, kennels and isolation units. Whether the organisation can take in the next animal depends on facts that change every day: which cages are free, which animals are still in medical care, which fosters have room at home and which treatments are due today.

In small and medium rescue groups these facts are usually kept in notebooks, messaging groups and personal spreadsheets maintained by different volunteers. Injuries are described verbally, treatment dates are remembered rather than scheduled, and the number of free cages is known only to the person who last cleaned them. As the number of animals grows, this informal record keeping leads to missed vaccinations, duplicated work and decisions made without a clear picture of capacity.

This project addresses that situation. It replaces scattered notes with a single Zoho Creator application that follows an animal from the moment it is rescued, through injury recording and ongoing treatment, to a cage in the shelter or a foster home, and that gives the administration a live view of capacity, animals in holding and treatments that are due.

1.2 Problem Statement

Rescue, rehabilitation and shelter administration are handled separately and mostly on paper or in informal chats, so the organisation cannot reliably answer basic operational questions: what condition was an animal in when it arrived, what treatment has it received and what is due next, who is fostering it, and whether there is a cage free for the next rescue. A system is needed that records each rescue and its injuries, schedules and tracks treatment, manages foster placements, and controls shelter capacity so that no cage is ever over-filled.

- Rescue details and the animal’s condition on arrival are recorded inconsistently, so staff who treat the animal later do not have the full history.

- Injuries and treatments are noted informally; follow-up treatments, vaccinations and medication schedules are missed because nothing reminds the vet.
- Foster carers are contacted by phone; there is no record of which animal is with which foster, for how long, or whether the foster still has room.
- Cage occupancy is not tracked, so animals are placed in full or unsuitable cages and animals in holding are forgotten.
- Administrators have no consolidated view of how many animals are in each state, how full the shelter is, or which cases are critical.
- All staff and volunteers see the same notes, so sensitive medical details cannot be restricted to the people who need them.

1.3 Objectives

- To design a normalised data model and forms in Zoho Creator for rescue reports, injuries, treatments, cages, holding assignments, foster carers and foster placements.
- To automate triage, escalation, cage-occupancy maintenance, foster validation and treatment reminders using Deluge scripting, form workflows and scheduled workflows.
- To prevent over-filling of cages and over-allocation of foster carers through validation rules that run before a record is saved.
- To provide role-based dashboards and reports for administrators, veterinarians, coordinators and caretakers, and a restricted self-service view for foster carers.
- To integrate the application with Zoho CRM so that foster carers and volunteers are managed as CRM contacts and can be selected through an Integration field.
- To deploy the application through Development, Stage and Production environments with record-level access control and governance controls.

1.4 Scope of the Project

In scope. The application supports six roles: Shelter Administrator, Veterinarian, Rescue Coordinator, Caretaker, Foster Carer and Volunteer. It covers the recording of rescue reports with the animal's condition and behaviour on arrival, injury records, treatment plans and logs, the master list of cages and enclosures, holding and cage assignments, the register of foster carers and foster placements. It also includes automatic status changes, alert e-mails,

scheduled treatment reminders, a daily capacity alert, release approval, dashboards and role-specific reports, and it is used through the Zoho Creator web interface and mobile app.

Out of scope. The application does not handle adoption, which is being developed separately by other students, nor donations, payments or veterinary billing. It does not integrate with government animal-welfare registries or with external veterinary clinic systems, and it does not use GPS tracking of rescue vehicles. Photographs are stored with records, but no image analysis is performed.

1.5 Justification for a Low-Code Approach

The shelter problem is a typical internal business process: a handful of related entities, clear rules, notifications and role-based visibility. Zoho Creator provides the building blocks for exactly this kind of application, namely forms with lookups, reports, a page builder, the Deluge scripting language, form and scheduled workflows, approvals, user permissions and native web and mobile apps, on a hosted platform [1]. Hosting, authentication, the database and the mobile client did not have to be built, which allowed the effort to go into the actual logic: cage-capacity control, foster validation and treatment reminders.

Low-code platforms are intended to let developers and domain experts build applications through visual modelling with only limited hand-written code [2], [3]. That suited this project, which was built by one student within a semester. Where visual tools alone were not enough, Zoho Creator still allowed targeted code: Deluge for server-side logic and reusable functions [4], and integration with Zoho CRM through the platform's connectors. The application is therefore maintainable through configuration for routine changes while still supporting the custom behaviour the problem requires.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements. The appendices give the feature coverage checklist, course outcome mapping,

the complete Deluge source code, a user manual, the evaluation rubric and the viva-voce record sheet.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

The existing process followed by a typical small rescue organisation works as follows:

1. A rescue request arrives by phone call or message; a volunteer notes the location and goes to collect the animal.
2. The rescuer records the animal's species, appearance and place of rescue in a notebook or on a personal phone, often without a photograph.
3. A veterinarian examines the animal; injuries and the treatment given are written on loose paper or told verbally to the caretaker.
4. The animal is placed in whichever cage appears to be free; the number of animals per cage is remembered by the staff on duty.
5. If the animal needs a quieter place to recover, a foster carer is found through phone calls, and the arrangement is noted informally.
6. Treatment dates, progress and the final outcome are updated later, if at all, so the record of each animal is incomplete.

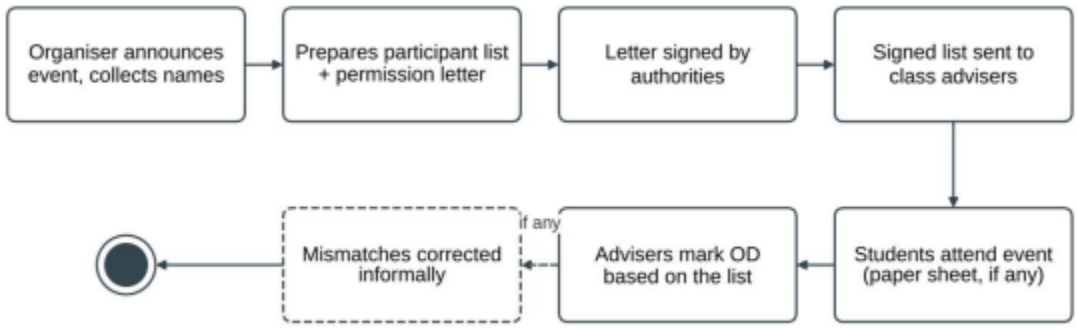


Figure 2.1: As-Is Process Flow of Manual Rescue and Shelter Record Keeping

2.2 Limitations of the Existing System

- Fragmented records: rescue details, injuries and treatments are spread across notebooks, chats and spreadsheets and cannot be searched together.

- Missed follow-ups: there is no scheduled reminder for the next dose, dressing change or vaccination.
- No capacity control: nothing prevents a cage from being over-filled, and animals in holding are easily forgotten.
- Untracked fosters: the organisation does not know how many animals each foster has, whether the foster accepts that species, or when the placement should end.
- No role-based access: every volunteer can see and change every record, including medical notes.
- No overview: management cannot see occupancy, critical cases or status counts without counting manually.

2.3 Low-Code versus Traditional Development

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Several weeks to a few months (estimate) for a database, back-end API, web front-end, mobile app, authentication and hosting	A few weeks of part-time work for the complete application including notifications and integration
Skills required	Full-stack development: back-end language and framework, database design, web and mobile front-end, DevOps	Zoho Creator configuration, Deluge scripting and basic API concepts
Hosting & infrastructure	Servers or cloud services must be provisioned, secured and maintained	Hosted by Zoho; Development, Stage and Production environments are available
Mobile application	A separate Android/iOS app would have to be built and published	The Zoho Creator mobile app runs the same application with camera and image upload
Security & access control	Authentication, roles and data protection must be implemented by hand	Built-in user management, roles, permission sets, record-level criteria and field permissions
Maintenance & change	Changes need code edits, testing and redeployment	Most changes are made in the builder and published through environments
Customisation limits	Unlimited; any UI or logic can be built	Limited to platform components; very custom UI needs widgets or external code
Cost	Hosting and developer time; no platform licence	Per-user subscription after the trial; no separate hosting cost

2.4 Literature Survey

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	A. Sahay, A. Indamutsa, D. Di Ruscio and A. Pierantonio, 2020 [5]	Supporting the understanding and comparison of low-code development platforms (SEAA 2020)	Proposes a feature-based comparison of popular low-code platforms covering data modelling, UI design, business logic, integration and deployment.	Provided the vocabulary used to map this project's features and to position Zoho Creator among low-code platforms.
2	A. C. Bock and U. Frank, 2021 [2]	Low-Code Platform (Business & Information Systems Engineering, 63(6))	Characterises low-code platforms and discusses citizen developers, organisational agility and development productivity.	Supports the justification that one student developer can deliver an operational process application.
3	D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi and M. Wimmer, 2022 [3]	Low-code development and model-driven engineering: Two sides of the same coin?	Compares low-code with model-driven engineering and identifies commonalities, differences, strengths and weaknesses.	Frames the forms, lookups and workflows used here as models from which Creator generates the running application.
4	M. A. A. Alamin et al., 2021 [6]	An empirical study of developer discussions on low-code software development challenges (arXiv:2103.11429)	Analyses about 5,000 Stack Overflow posts on low-code platforms; customisation and documentation gaps are recurring challenges.	Matches the platform limits met in this project, such as restrictions on fetch criteria and report filtering.
5	Zoho Corporation [1], [4]	Zoho Creator Help and Deluge Reference Guide	Official documentation of forms, workflows, schedules, permissions, approvals and environments.	Primary technical reference for every component implemented in Chapter 5.
6	Zoho Corporation [8]	Zoho CRM Developer Documentation	Describes the CRM modules, records and APIs through which Creator exchanges data with Zoho CRM.	Basis of the CRM integration used for foster carers and volunteers.

2.5 Emerging Trends in Low-Code Development

Three trends in low-code development shaped this project. The first is AI-assisted application building. Zoho Creator includes Zia, its AI assistant, which can generate application drafts and provides AI fields; in this project AI assistance was also used outside the platform to plan the design and draft scripts, and every script was entered, tested and corrected by the student in the Creator editor (Section 5.9).

The second trend is citizen development combined with pro-code extension. Platforms increasingly let non-specialists build most of an application visually while professional developers extend it where necessary [2], [3]. The Pet Rescue System follows that pattern:

forms, reports and dashboards are configured visually, while shared logic such as cage recounting is written once as Deluge functions.

The third trend is composable, integrated applications. Rather than standing alone, low-code applications connect to CRM, messaging and e-mail services through managed connections. Here, Zoho Creator and Zoho CRM work together so that foster carers and volunteers can be managed as CRM contacts and selected from inside the shelter application.

2.6 Proposed System

The Pet Rescue System is a Zoho Creator application that manages rescue, rehabilitation and shelter administration in one place. It addresses each limitation of the existing system as follows. Fragmented records are replaced by linked forms in which every injury, treatment, cage assignment and foster placement refers to one animal record. Missed follow-ups are prevented by a scheduled workflow that e-mails the veterinarian a digest of treatments due, and a declining animal or a critical arrival triggers an immediate alert. Capacity is controlled by validation workflows that refuse an assignment to a full or unavailable cage, and by a function that recounts occupancy after every change. Foster placements are validated against the foster's availability, capacity and accepted species. Role-based permissions limit each user to the forms, fields and records they need, and dashboards give administrators a live picture of occupancy, animals by status and treatments due.

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Shelter Administrator	The shelter manager or application owner	Manages cages, users and permissions; approves releases; views all dashboards and reports; receives capacity alerts
Veterinarian	Vet or paravet treating the animals	Records injuries and treatment plans; reviews critical and declining cases; receives treatment reminders
Rescue Coordinator	Operations staff coordinating rescues	Creates rescue reports; assigns cages; registers foster carers and creates foster placements
Caretaker	Shelter attendant handling daily care	Logs treatments administered; assigns and moves animals between cages; views animal status
Foster Carer	External volunteer fostering animals at home	Views own fostered animals; logs care and check-ins; updates own placement status
Volunteer	General helper	Reports a rescue; views cage availability and holding register

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall record a rescue with date and time, location, rescue type, species, condition and behaviour on arrival, and a photograph.	Coordinator, Volunteer	High
FR-02	The system shall generate a unique Animal ID and set the initial status from the arrival condition.	System	High
FR-03	The system shall alert the veterinarian by e-mail when an animal arrives in critical condition.	System	High
FR-04	The system shall record one or more injuries per animal with type, body part, severity and recovery status.	Vet, Coordinator	High
FR-05	The system shall escalate an animal to medical care and notify the vet when a severe injury is recorded.	System	Medium
FR-06	The system shall record treatments with type, medication, dosage, weight, progress and the next treatment due date.	Vet, Caretaker	High
FR-07	The system shall reject a next-treatment date earlier than the treatment date and an injury date in the future.	System	Medium
FR-08	The system shall alert the vet when a treatment log marks an animal’s progress as Declining.	System	Medium

FR ID	Requirement	Role(s)	Priority
FR-09	The system shall maintain a master list of cages with type, location, capacity and status.	Administrator	High
FR-10	The system shall refuse an assignment to a cage that is full, out of service or under cleaning.	System	High
FR-11	The system shall recompute cage occupancy and status automatically after every assignment, move or release.	System	High
FR-12	The system shall register foster carers with capacity, accepted species and availability.	Coordinator	High
FR-13	The system shall validate a foster placement against the foster's availability, maximum animals and accepted species.	System	High
FR-14	The system shall, on a valid placement, set the animal to With Foster, free its cage and update the foster's availability.	System	High
FR-15	The system shall e-mail the vet a daily digest of treatments due, and the administrator an alert when occupancy reaches 90%.	System	Medium
FR-16	The system shall route release of an animal for administrator approval.	Administrator	Medium
FR-17	The system shall show role-specific dashboards and reports, and foster carers only their own animals and placements.	All roles	High

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Validation and status updates shall complete within the same form submission; occupancy shall be recomputed by counting open assignments rather than by incremental updates.
NFR-02	Security	Users shall access only the forms, fields and records permitted by their permission set and record-level criteria; medical fields shall be hidden from volunteers.
NFR-03	Usability	The application shall be usable on the Zoho Creator web interface and mobile app; a rescue shall be recordable in a single short form.
NFR-04	Availability	The application shall be hosted on Zoho's cloud and remain accessible wherever an internet connection is available.
NFR-05	Maintainability	Shared logic (cage recount, foster availability, closing assignments) shall be implemented as reusable Deluge functions; fields shall use consistent link names.
NFR-06	Auditability	Workflow and schedule executions shall be traceable through the application logs, and every record shall retain its added user and time.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator (free trial with premium features)

Category	Specification
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox (latest version)
Other Zoho services	Zoho CRM (foster carers and volunteers as Contacts); Zoho Creator e-mail (sendmail)
API testing tool	Postman (for Creator REST API v2.1 tests)
Mobile	Zoho Creator mobile app (Android / iOS) for caretakers, foster carers and volunteers, with camera for animal and injury photographs
Client hardware	Intel Core i3/i5 or above, 8 GB RAM, internet connection

3.5 Use Case Model

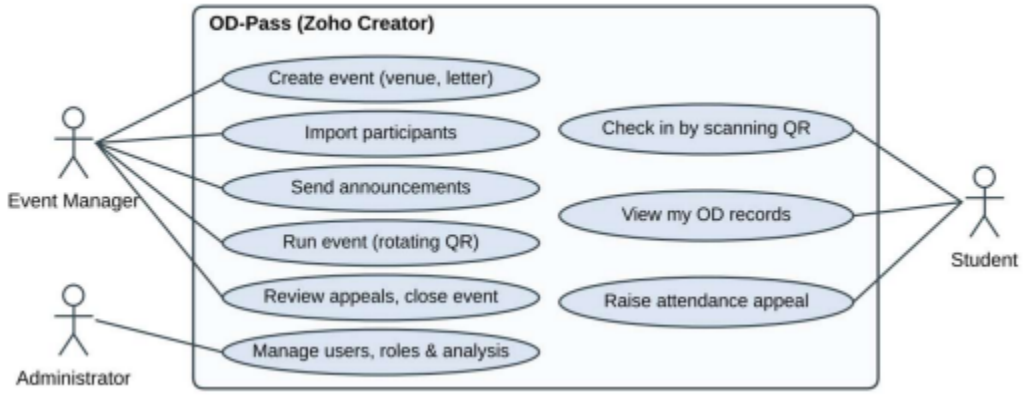


Figure 3.1: Use Case Diagram of the Pet Rescue System

Table 3.5: Use Case Description – Assign an Animal to a Cage

Item	Description
Use Case ID	UC-01
Use Case Name	Assign an animal to a cage
Actor(s)	Rescue Coordinator or Caretaker (primary), System
Pre-condition	The animal has a Rescue Report; the cage exists in the Cage Master; the user has permission on the Holding Assignment form.
Main Flow	1. The user opens the Holding Assignment form and selects the animal, the cage, the holding reason and the expected move date. 2. On validation, the system checks that the cage is neither Out of Service nor Under Cleaning. 3. The system checks that current occupancy is below capacity. 4. On success, the system closes any other open assignment of the animal. 5. The system recounts occupancy of the affected cages and sets their status to Full or Available. 6. The animal's Current Cage and status are updated.
Alternate Flow	2a. Cage unavailable: an alert is shown and the submission is cancelled. 3a. Cage full: an alert is shown and the submission is cancelled.

Item	Description
Post-condition	An open assignment exists for the animal; the cage occupancy and status are correct.
Related FR	FR-10, FR-11

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

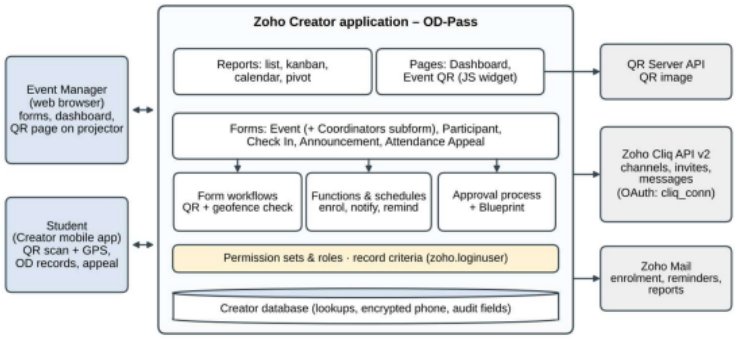


Figure 4.1: System Architecture of the Pet Rescue System

The application is organised in four layers. The access layer consists of the users and their channels: administrators, veterinarians and coordinators work mainly on the Zoho Creator web interface, while caretakers, foster carers and volunteers use the Zoho Creator mobile app, which also provides the camera for animal and injury photographs. The application layer is the Zoho Creator application itself: seven forms, eleven reports, two pages, twelve form workflows, three shared Deluge functions, two schedules and an approval process. Permission sets and record-level criteria at this layer decide what each user can see.

The data layer is Zoho Creator's managed database, in which the forms are linked by lookup fields; system fields such as Added User and Added Time are stored automatically with every record, and photographs are stored as image fields. The integration layer connects the application to other services: Zoho CRM for foster carers and volunteers, Zoho Creator's e-mail service through sendmail, and the Creator REST API v2.1 for access from outside the application [9].

4.2 Data Model

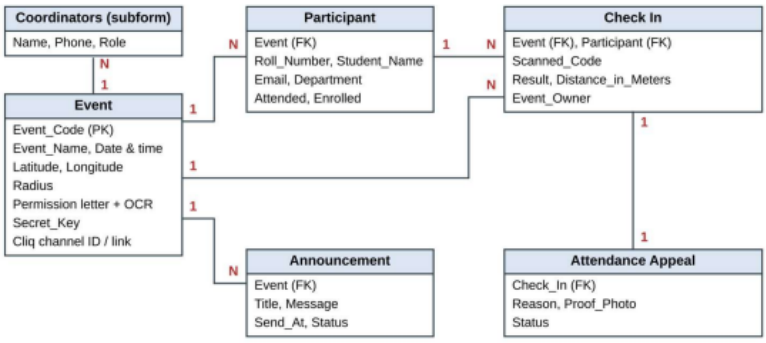


Figure 4.2: Entity–Relationship Diagram of the Pet Rescue System

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship	Implemented Using
Rescue_Report	Animal_ID, Species, Rescue_Type, Condition_on_Arrival, Current_Status	Injury_Record	One-to-Many	Lookup (Animal) in Injury_Record
Rescue_Report	as above	Treatment_Log	One-to-Many	Lookup (Animal) in Treatment_Log
Rescue_Report	as above	Holding_Assign ment	One-to-Many	Lookup (Animal) in Holding_Assignment
Rescue_Report	as above	Foster_Placement	One-to-Many	Lookup (Animal) in Foster_Placement
Cage_Master	Cage_ID, Cage_Type, Capacity, Current_Occupancy, Cage_Status	Holding_Assign ment	One-to-Many	Lookup (Cage) in Holding_Assignment
Cage_Master	as above	Rescue_Report	One-to-Many	Lookup (Current_Cage), set by script
Foster_Master	Foster_ID, Species_Accepted, Max_Animals, Availability	Foster_Placement	One-to-Many	Lookup (Foster) in Foster_Placement

The Rescue Report is the centre of the model. Injuries, treatments, cage assignments and foster placements all refer to an animal through lookup fields, which allows the animal's detail view to show its complete history as related blocks. A cage is linked to many assignments, and its occupancy is not typed in by a user but recomputed by counting the open assignments (those without a moved-out date). Two values are deliberately denormalised for convenience: Current_Cage on the Rescue Report records where the animal is now, and Foster_Email on the Foster Placement stores the carer's login e-mail so that record-level criteria can compare it with the logged-in user.

4.3 Form Design

Table 4.2: Field Specification – Cage Master Form

Field Label	Link Name	Field Type	Mandatory	Validation / Remarks
Cage ID	Cage_ID	Auto Number	—	Prefix CG-; unique cage identifier
Cage Type	Cage_Type	Drop Down	Yes	Dog Kennel, Cat Cage, Isolation, Quarantine, Medical Ward, Holding Pen
Block / Location	Block_Location	Single Line	No	Position inside the shelter
Capacity	Capacity	Number	Yes	Minimum 1; maximum animals the cage can hold
Current Occupancy	Current_Occupancy	Number	—	Default 0; set by updateCageOccupancy; read-only for users
Cage Status	Cage_Status	Drop Down	Yes	Available, Full, Under Cleaning, Out of Service; Full / Available set by script

Table 4.3: Field Specification – Rescue Report Form

Field Label	Link Name	Field Type	Mandatory	Validation / Remarks
Animal ID	Animal_ID	Auto Number	—	Prefix AN-; unique animal identifier
Animal Name	Animal_Name	Single Line	No	Default “Unknown”
Species	Species	Drop Down	Yes	Dog, Cat, Bird, Cow, Monkey, Other
Breed / Description	Breed_Description	Single Line	No	Colour, markings, breed
Approx. Age / Sex	Approx_Age, Sex	Drop Down / Radio	No	Newborn–Senior; Male, Female, Unknown
Photo	Photo	Image	No	Camera or gallery
Rescue Date & Time	Rescue_Date_Time	Date-Time	Yes	Default: current time (on load)
Rescue Location	Rescue_Location	Single Line	Yes	Place of rescue
Reported By / Contact	Reported_By, Reporter_Contact	Single Line / Phone	No	Informant details
Rescue Type	Rescue_Type	Drop Down	Yes	Road Accident, Abandoned, Abuse, Stray, Natural Disaster, Surrender
Condition on Arrival	Condition_on_Arrival	Drop Down	Yes	Critical, Serious, Stable, Healthy; drives triage
Behaviour on Arrival	Behaviour_on_Arrival	Drop Down	No	Calm, Scared, Aggressive, Unresponsive
Microchipped	Microchipped	Radio	No	Yes / No
Current Status	Current_Status	Drop Down	Yes	In Holding, In Medical Care, In Rehab, With Foster, Ready for Release, Released, Deceased
Current Cage	Current_Cage	Lookup	—	Cage_Master; set by script

Table 4.4: Field Specification – Injury Record Form

Field Label	Link Name	Field Type	Mandatory	Validation / Remarks
Animal	Animal	Lookup	Yes	Rescue_Report; display Animal ID – Name
Injury Date Noted	Injury_Date_Noted	Date	Yes	Not in the future (validation)
Injury Type	Injury_Type	Drop Down	Yes	Fracture, Open Wound, Burn, Infection, Malnutrition, Parasites, Neurological, Other
Body Part Affected	Body_Part_Affected	Drop Down	Yes	Head, Front Leg, Hind Leg, Torso, Tail, Eyes, Multiple
Severity	Severity	Drop Down	Yes	Mild, Moderate, Severe; Severe triggers escalation
Description / Photo	Description, Injury_Photo	Multi Line / Image	No	Assessment notes and injury photograph
Assessed By	Assessed_By	Single Line	No	Vet or paravet
Recovery Status	Recovery_Status	Drop Down	Yes	Untreated, Under Treatment, Recovering, Healed

Table 4.5: Field Specification – Treatment Log Form

Field Label	Link Name	Field Type	Mandatory	Validation / Remarks
Animal	Animal	Lookup	Yes	Rescue_Report
Related Injury	Related_Injury	Lookup	No	Injury_Record for the same animal
Treatment Date	Treatment_Date	Date	Yes	Default: today
Treatment Type	Treatment_Type	Drop Down	Yes	Medication, Surgery, Dressing, Vaccination, Deworming, Physiotherapy, Checkup
Medication / Dosage	Medication_Procedure, Dosage	Single Line	No	Hidden from volunteers by field permissions
Vet Notes	Vet_Notes	Multi Line	No	Hidden from volunteers
Weight (kg)	Weight_kg	Decimal	No	Tracks recovery
Next Treatment Due	Next_Treatment_Due	Date	No	Not before treatment date; drives reminders and calendar
Progress	Progress	Drop Down	Yes	Improving, Stable, Declining
Follow-up Done	Follow_up_Done	Decision Box	—	Default unchecked; set by script

Table 4.6: Field Specification – Foster Master Form

Field Label	Link Name	Field Type	Mandatory	Validation / Remarks
Foster ID	Foster_ID	Auto Number	—	Prefix FS-
Foster Name	Foster_Name	Name	Yes	First and last name
Email / Mobile	Email, Mobile	Email / Phone	Yes	Email must match the carer's login e-mail; stored as personal data

Field Label	Link Name	Field Type	Mandatory	Validation / Remarks
Address	Address	Address	No	Home address of the carer
Species Accepted	Species_Accepted	Multi Select	Yes	Same options as the Species drop down
Max Animals	Max_Animals	Number	Yes	Capacity at the carer's home
Experience Level	Experience_Level	Drop Down	No	Beginner, Experienced, Medical-trained
Availability	Availability	Drop Down	Yes	Available, Currently Fostering, On Break; refreshed by script

Table 4.7: Field Specification – Foster Placement Form

Field Label	Link Name	Field Type	Mandatory	Validation / Remarks
Animal	Animal	Lookup	Yes	Rescue_Report
Foster	Foster	Lookup	Yes	Foster_Master; Integration field to a CRM contact where used
Placement Start Date	Placement_Start_Date	Date	Yes	Default: today
Expected End Date	Expected_End_Date	Date	No	Not before start date (validation)
Reason for Fostering	Reason_for_Fostering	Drop Down	Yes	Recovery, Too Young, Overcrowding, Behavioural Rehab
Care Instructions	Care_Instructions	Multi Line	No	Feeding, medication, handling
Check-in Date / Condition	Check_in_Date, Animal_Condition_at_Check_in	Date / Drop Down	No	Improving, Stable, Concern
Placement Status	Placement_Status	Drop Down	Yes	Active, Completed, Returned Early; default Active
Foster Email	Foster_Email	Email	—	Hidden; set by script for record-level criteria

Table 4.8: Field Specification – Holding Assignment Form

Field Label	Link Name	Field Type	Mandatory	Validation / Remarks
Animal	Animal	Lookup	Yes	Rescue_Report
Cage	Cage	Lookup	Yes	Cage_Master; display Cage ID – Type
Assigned Date	Assigned_Date	Date	Yes	Default: today
Holding Reason	Holding_Reason	Drop Down	Yes	New Intake, Quarantine, Medical Observation, Awaiting Foster
Expected Move Date	Expected_Move_Date	Date	No	Planned date of moving out
Moved Out Date	Moved_Out_Date	Date	No	Blank while the animal is still in the cage

4.4 Process and Workflow Design

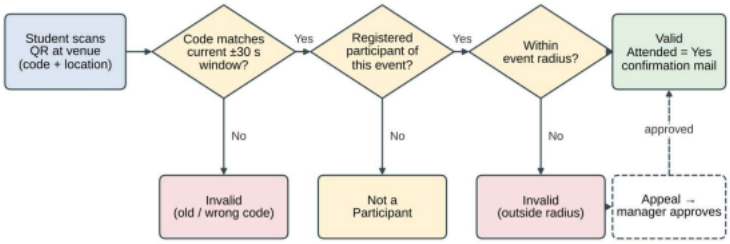


Figure 4.3: Process Flow for Cage Assignment Validation

The central automated process is cage assignment. When staff submit the Holding Assignment form, the validation workflow reads the selected cage and stops the submission if the cage is out of service, under cleaning or already at capacity. When the assignment is accepted, the success workflow closes any other open assignment of the same animal, calls the shared function that recounts occupancy and sets the cage to Full or Available, and updates the animal's current cage and status.

The other processes follow the same pattern of a trigger followed by a Deluge action. Creating a rescue report sets the initial status and alerts the vet for critical arrivals; a severe injury or a declining treatment log escalates the animal to medical care; a foster placement is validated, then moves the animal out of its cage and refreshes the foster's availability; a daily schedule sends the treatment digest, a second schedule checks overall occupancy, and an approval process governs release.

4.5 User Interface and Navigation Design

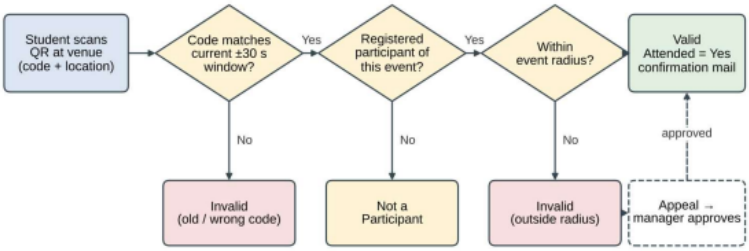


Figure 4.4: Navigation Structure of the Pet Rescue System

The navigation menu has five entries: Rescue (Rescue Report, All Animals, Animal Status Board, Injury Record), Rehabilitation (Treatment Log, Treatment History, Treatment

Calendar, Foster Master, Foster Placement), Shelter (Cage Master, Cage Status, Holding Assignment, Holding Register, Occupancy Chart), Dashboard and My Fostered Animals. Because Zoho Creator hides any component a user has no permission for, the same menu structure produces a different, smaller menu for each role: a foster carer sees only the foster section, while the administrator sees every entry.

4.6 Access Control Design

Table 4.9: Role–Permission Matrix

Form	Admin	Vet	Coordinator	Caretaker	Foster	Volunteer
Cage Master	CRUD	R	R	R	—	R
Rescue Report	CRUD	R U	C R U	R	R (own)	C R
Injury Record	CRUD	C R U	C R	C R	R (own)	—
Treatment Log	CRUD	C R U	R	C R	C R (own)	—
Foster Master	CRUD	R	C R U	—	R (own)	—
Foster Placement	CRUD	R	C R U	R	R U (own)	—
Holding Assignment	CRUD	R	C R U	C R U	—	R

C = Create, R = Read, U = Update, D = Delete, (own) = only records linked to the logged-in user, — = no access.

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application was created on Zoho Creator in the India data centre (creator.zoho.in) with the link name petscop and the time zone Asia/Kolkata. Components are grouped into navigation sections for each role (Figure 4.4). The application was then added to Zoho Creator Environments so that it has separate Development and Stage versions before publishing to Production [7].

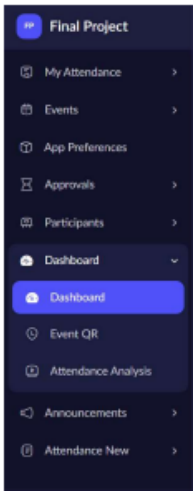


Figure 5.1: Application Home – Navigation Sidebar with the Rescue, Rehabilitation and Shelter Sections

5.2 Forms and Data Capture

The application has seven forms (Tables 4.2–4.8). They use more than eight field types: Single Line, Multi Line, Number, Decimal, Date, Date-Time, Email, Phone, Name, Address, Image, Decision Box, Drop Down, Radio, Multi Select, Lookup and Auto Number.

Cage Master defines every cage, with its capacity and a status that scripts keep accurate. **Rescue Report** is the main animal record; an on-load workflow defaults the status and the rescue time, and the arrival condition drives the starting status. **Injury Record** and **Treatment Log** refer to the animal by lookup; the Treatment Log carries the Next Treatment Due date that feeds the reminder schedule and the calendar report. **Foster Master** stores each carer's capacity and accepted species, and **Foster Placement** links an animal to a carer for a period. **Holding Assignment** records which animal is in which cage and since when. Fields

set by scripts, such as Current Occupancy and Follow-up Done, are made read-only or hidden so that users cannot tamper with them.

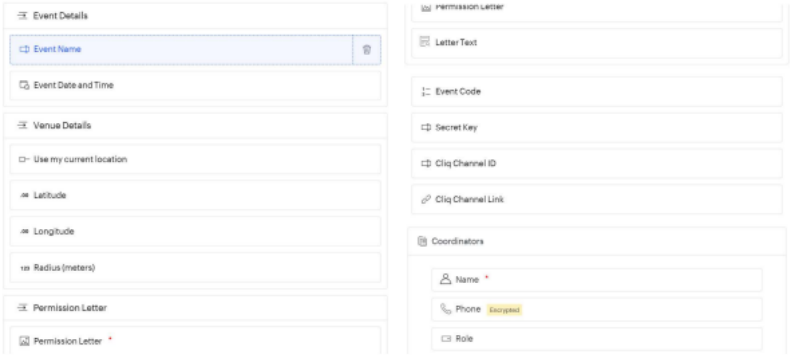


Figure 5.2: Rescue Report Form in the Form Builder

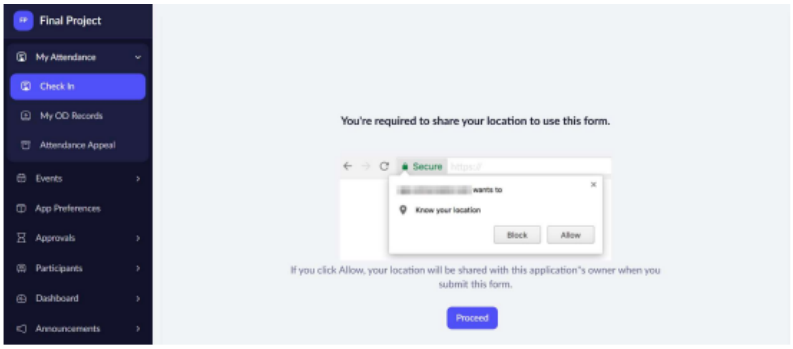


Figure 5.3: Holding Assignment Form in Live Mode – Cage Full Alert

5.3 Reports

Table 5.1: Reports Implemented

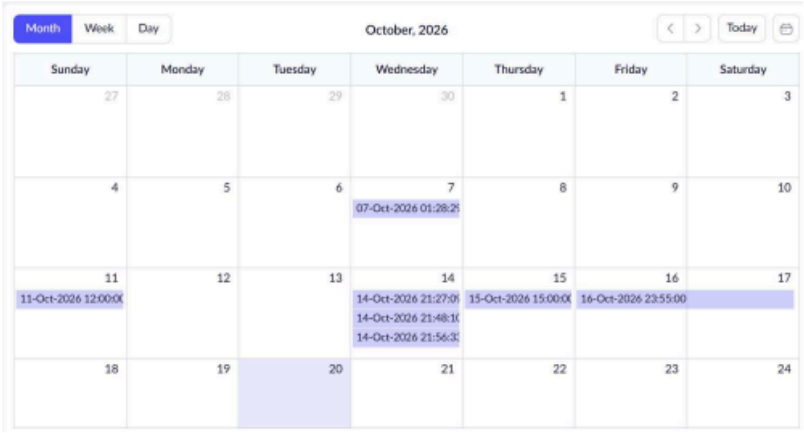
Report Name	Type	Source Form	Filter / Grouping	Roles
All Animals	List	Rescue_Report	Grouped by Current_Status; conditional formatting on Condition_on_Arrival	Admin, Vet, Coordinator, Caretaker
Animal Status Board	Kanban	Rescue_Report	Columns by Current_Status	Admin, Coordinator, Caretaker
Injury Report	List	Injury_Record	Grouped by Severity	Admin, Vet, Coordinator
Treatment History	List	Treatment_Log	Grouped by Animal; sorted by Treatment_Date	Admin, Vet, Caretaker
Treatment Calendar	Calendar	Treatment_Log	Date: Next_Treatment_Due; Follow_up_Done == false	Admin, Vet
Cage Status	List	Cage_Master	Conditional colours for Full and Out of Service	All staff
Occupancy Chart	Pivot Chart	Cage_Master	Bars by Cage_Type; sum of Capacity and Current_Occupancy	Admin, Coordinator

Report Name	Type	Source Form	Filter / Grouping	Roles
Foster Directory	List	Foster_Master	Grouped by Availability	Admin, Coordinator, Vet
Foster Placements	List	Foster_Placement	Grouped by Placement_Status	Admin, Coordinator, Vet
Holding Register	List	Holding_Assign ment	Moved_Out_Date == null; grouped by Cage	Admin, Coordinator, Caretaker, Volunteer
My Fostered Animals	List	Foster_Placement	Foster_Email == zoho.loginuserid; Placement_Status == Active	Foster Carer

Eleven reports of four types were built: eight list reports, one kanban, one calendar and one pivot chart. Reports that a foster carer sees carry record-level criteria, so the same report shows different data to different users. Figure 5.4 shows the Animal Status Board and the Treatment Calendar with test data.



(a) Check-In Kanban grouped by Result



(b) Treatment Schedule (October 2026)

Figure 5.4: Animal Status Board (Kanban) and Treatment Calendar Reports

5.4 Pages and Dashboards

Two pages were built with the UI Builder. The Shelter Dashboard is the home screen for administrators, coordinators and veterinarians. Its first row has four KPI panels: Animals in Care, Critical Cases, Cages Full and Treatments Due This Week. The second row has a column chart of cage occupancy against capacity and a donut chart of animals by status. The

third row embeds the Treatment Calendar and buttons to add a rescue report or a holding assignment. The My Foster Dashboard shows a foster carer the animals currently in their care and the next check-in date, using criteria that restrict every element to the logged-in carer.

5.5 Deluge Scripting

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Rescue Report → On Load	Default status and rescue time	input assignment, zoho.currenttime
DS-02	Rescue Report → On Success (created)	Set triage status; alert vet for critical animals	if, sendmail
DS-03	Rescue Report → On Success (edited)	Free cages when released or deceased	function call
DS-04	Injury Record → On Validate	Reject future injury dates	date comparison, alert, cancel submit
DS-05	Injury Record → On Success	Escalate severe injuries; notify vet	fetch / update, sendmail
DS-06	Treatment Log → On Validate	Check next-treatment date	alert, cancel submit
DS-07	Treatment Log → On Success	Declining alert; close earlier follow-ups	for each, fetch / update
DS-08	Holding Assignment → On Validate	Block full or unavailable cages	fetch, nested if, alert
DS-09	Holding Assignment → On Success	Close old assignments; recount; sync animal	collections, function calls
DS-10	Foster Placement → On Validate	Capacity, availability and species checks	count(), contains()
DS-11	Foster Placement → On Success (created)	Move animal to foster; free cage; store foster e-mail	function calls, input assignment
DS-12	Foster Placement → On Success (edited)	Completion or early return; concern alert	if / else if, sendmail
DS-13	Function: updateCageOccupancy(cageId)	Recount open assignments and set cage status	criteria count, update
DS-14	Function: refreshFosterAvailability(fosterId)	Recompute foster availability	criteria count, update
DS-15	Function: closeCageAssignments(animalId)	Close all open cage assignments of an animal	for each, function call
DS-16	Schedule: treatment reminders	Daily digest of treatments due	string building, sendmail
DS-17	Schedule: capacity alert	Alert at 90% occupancy	sum(), ifnull, round
DS-18	Approval: release approval	Notify on approval; revert status on rejection	fetch / update, sendmail

Code 5.1: Cage Recount Function (DS-13)

```
// DS-13 | Function: updateCageOccupancy(int cageId)
cageRec = Cage_Master[ID == cageId];
occ = Holding_Assignment[Cage == cageId && Moved_Out_Date == null].count();
cageRec.Current_Occupancy = occ;
```

```
if(cageRec.Cage_Status != "Out of Service" && cageRec.Cage_Status != "Under Cleaning")
{
    if(occ >= cageRec.Capacity)
    {
        cageRec.Cage_Status = "Full";
    }
    else
    {
        cageRec.Cage_Status = "Available";
    }
}
```

Code 5.2: Cage Assignment Validation (DS-08)

```
// DS-08 | Form: Holding Assignment | Event: On Validate
cageRec = Cage_Master[ID == input.Cage];
if(cageRec.Cage_Status == "Out of Service" || cageRec.Cage_Status == "Under Cleaning")
{
    alert "This cage is " + cageRec.Cage_Status + ". Choose another cage.";
    cancel submit;
}
if(cageRec.Current_Occupancy >= cageRec.Capacity)
{
    alert "Cage " + cageRec.Cage_ID + " is full. Choose another cage.";
    cancel submit;
}
```

Code 5.1 is the core of the shelter logic. Instead of adding or subtracting one from a stored occupancy figure, which drifts whenever a record is edited or deleted, the function recounts the open assignments of the cage (those without a moved-out date) every time it is called, and then sets the cage status. The function is called from the holding assignment workflow, the release workflow and the foster placement workflow, so the rule is written once. Code 5.2 shows the validation that uses the stored occupancy to refuse a full or unavailable cage before the record is saved. The complete source of every script is given in Appendix C.

5.6 Workflows and Automation

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Action(s)	Script Ref.
Rescue Defaults	Form Workflow	Rescue Report: on load (create)	Default status and rescue time	DS-01
Triage and Critical Alert	Form Workflow	Rescue Report: on success (create)	Set status; e-mail vet if Critical	DS-02
Release Housekeeping	Form Workflow	Rescue Report: on success (edit)	Close cage assignments if Released / Deceased	DS-03
Validate Injury	Form Workflow	Injury Record: on validate	Block future injury date	DS-04
Escalate Severe Injury	Form Workflow	Injury Record: on success	Set In Medical Care; notify vet	DS-05

Workflow Name	Type	Trigger / Event	Action(s)	Script Ref.
Validate Treatment	Form Workflow	Treatment Log: on validate	Check next-due date	DS-06
Treatment Follow-up	Form Workflow	Treatment Log: on success	Declining alert; close old follow-ups	DS-07
Validate Cage	Form Workflow	Holding Assignment: on validate	Block full / unavailable cage	DS-08
Sync Cage Assignment	Form Workflow	Holding Assignment: on success (create / edit)	Close old assignments; recount; update animal	DS-09
Validate Foster	Form Workflow	Foster Placement: on validate (create)	Capacity, availability and species checks	DS-10
Place with Foster	Form Workflow	Foster Placement: on success (create)	Update animal, cage, foster availability	DS-11
Foster Status Change	Form Workflow	Foster Placement: on success (edit)	Completion / early return; concern alert	DS-12
Treatment Reminders	Schedule	Daily, 08:00 IST	E-mail digest of treatments due	DS-16
Capacity Alert	Schedule	Daily, 09:00 IST	E-mail admin at 90% occupancy	DS-17
Release Approval	Approval	Rescue Report: status Ready for Release	Route to Shelter Administrator; approve / reject actions	DS-18

In total there are twelve form workflows, two schedules and one approval process. A Blueprint and a batch workflow were not needed: the status field with its workflows already enforces the animal's lifecycle, and recounting is done per cage rather than in bulk. The approval gives the Shelter Administrator control over which animals leave the shelter, while the schedules ensure that routine reminders do not depend on anyone remembering them.

5.7 Integrations

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Zoho CRM contacts	Zoho CRM – Contacts module	Integration field in Creator	Zoho CRM connection	Foster carers and volunteers are selected from CRM contacts on the Foster Placement form
Foster sync to CRM	Zoho CRM – create record	Deluge zoho.crm.createRecord	Zoho CRM connection	New Foster Master record creates a matching CRM contact
Alert e-mails	Zoho Creator e-mail	sendmail	Application owner account	Critical arrivals, severe injuries, declining animals, reminders
Creator REST API	Creator REST API v2.1 (zohoapis)	HTTP GET	OAuth 2.0 (Self Client)	Read report records from outside, tested from Postman

Code 5.3: Creating a CRM Contact for a New Foster Carer (excerpt)

```
// Form: Foster Master | Event: On Success (created)
contactMap = Map();
contactMap.put("First_Name", input.Foster_Name.first_name);
contactMap.put("Last_Name", input.Foster_Name.last_name);
contactMap.put("Email", input.Email);
contactMap.put("Mobile", input.Mobile);
resp = zoho.crm.createRecord("Contacts", contactMap);
info resp;
```

Foster carers and volunteers are people the organisation keeps in touch with over time, which is the kind of relationship Zoho CRM is designed to manage. When a carer is added in Creator, the script in Code 5.3 creates the matching CRM contact, and on the Foster Placement form an Integration field lets staff select an existing contact pulled directly from CRM. The Zoho CRM connection is created once under Settings → Connections and enabled only for this application, so no keys appear in any script. The Creator REST API was also tested from outside the application: a Self Client was created in the Zoho API Console, a grant code with a read scope was exchanged for an access token, and a GET request for a report was sent from Postman with the token in the Authorization header [9].

5.8 User Management and Role-Based Access

Access control was implemented as designed in Table 4.9. Roles were created for the Shelter Administrator, Veterinarian, Rescue Coordinator, Caretaker, Foster Carer and Volunteer, and one permission set was created for each. Users were added from the Users page of the Creator account and given both a role and a permission set for the application; a test user was created for each role.

Each permission set grants the form, report and page access shown in the matrix. Field-level permissions hide Vet Notes, Dosage and Medication from volunteers, and make Cage Status, Current Occupancy and Current Status read-only for everyone except administrators and coordinators, since scripts maintain them. Foster carers are separated from other users by record-level criteria rather than by ownership: the Foster Placement form stores the carer's e-mail in a hidden field, and the criteria Foster_Email == zoho.loginuserid limit every report and dashboard element to that carer's own placements. A carer therefore sees only the animals currently in their care.

5.9 AI and Analytics Features

AI features: Zoho Creator offers Zia, its AI assistant, for generating applications and for AI fields such as OCR. In this project, an AI assistant (Claude) was used during development to help plan the data model, draft Deluge code and workflows and explain errors; every script was entered, tested and corrected by the student in the Creator editor, and the design decisions were reviewed by the student. This is declared here in line with the declaration at the beginning of the report.

Analytics: The Shelter Dashboard provides KPI panels, an occupancy chart and a donut chart of animals by status. The Occupancy Chart pivot report compares capacity with current occupancy for each cage type, and the Animal Status Board groups animals by status for quick review. Because occupancy is recomputed by scripts, these figures are always current.

5.10 Pro-Code Extension

The Pet Rescue System is built almost entirely with configuration and Deluge. The three shared functions (updateCageOccupancy, refreshFosterAvailability and closeCageAssignments) are the main extension beyond visual tools: they hold logic that several workflows reuse, and they keep that logic out of the forms. The Creator REST API v2.1 allows external programs to read and write records, which would be the route for a separate public website or mobile client [9]. No JavaScript widget or Zoho Catalyst function was needed, because nothing in the requirements exceeded what Deluge could do. A cage map drawn as a widget is listed as a future enhancement.

5.11 Mobile and Portal Access

Caretakers, foster carers and volunteers use the Zoho Creator mobile app. After accepting the invitation and signing in, each user sees only the sections their permission set allows. Volunteers can submit a rescue report with a photograph taken on the phone, caretakers can assign cages and log treatments at the cage, and foster carers can open My Fostered Animals and record check-ins from home. No customer portal was used, because every user is added as a regular application user; a portal would be the next step if the number of external foster carers grows.

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing is carried out at four levels. Unit testing checks each form and script in isolation: each workflow is saved and triggered from live mode, and the shared functions are run with the Execute option in the function editor with test arguments. Integration testing covers the paths that cross components: a rescue report to a cage assignment, a severe injury to the animal's status and the vet's alert, a foster placement to the animal, cage and foster records, and a new foster carer to the CRM contact.

Role-based access testing is done by logging in with a separate test account for each role (Administrator, Veterinarian, Coordinator, Caretaker, Foster Carer and Volunteer) on separate browsers and on the mobile app, and checking what each can see and do. User acceptance testing is recorded in Section 6.5. Every test case is traced to a functional requirement in Table 6.1, and the Actual Result and Status columns are completed as each test is executed.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description and Steps	Expected Result	Status
TC-01	FR-01, FR-02	Submit a rescue report: Fill the Rescue Report form with a Stable dog and submit	Record saved with a unique Animal ID; status In Holding	PASS
TC-02	FR-03	Critical arrival alert: Submit a report with condition Critical	Status In Medical Care; e-mail received by vet	PASS
TC-03	FR-04, FR-05	Severe injury escalation: Add an injury with severity Severe for a stable animal	Animal status In Medical Care; vet e-mail received	PASS
TC-04	FR-07	Future injury date: Enter an injury date after today	Alert shown; submission blocked	PASS
TC-05	FR-06	Log a treatment: Submit a treatment with next due date	Record saved; earlier follow-ups marked done	PASS
TC-06	FR-07	Invalid next-due date: Next due date earlier than treatment date	Alert shown; submission blocked	PASS
TC-07	FR-08	Declining progress alert: Log a treatment with progress Declining	Vet e-mail received; status In Medical Care	PASS
TC-08	FR-09	Create a cage: Add a cage with capacity 2	Cage saved; occupancy 0; status Available	PASS
TC-09	FR-10, FR-11	Assign to a free cage: Assign an animal to the cage	Occupancy 1; animal's Current Cage set	PASS
TC-10	FR-10, FR-11	Fill a cage: Assign a second animal to the same cage	Occupancy 2; cage status Full	PASS

TC ID	FR Ref.	Test Description and Steps	Expected Result	Status
TC-11	FR-10	Assign to a full cage: Assign a third animal to the full cage	“Cage is full” alert; submission blocked	PASS
TC-12	FR-10	Assign to an unavailable cage: Set cage Under Cleaning and assign an animal	Alert shown; submission blocked	PASS
TC-13	FR-11	Move an animal: Assign an animal already in a cage to another cage	Old assignment closed; both cages recounted	PASS
TC-14	FR-12	Register a foster: Add a foster accepting dogs, maximum 1	Record saved; availability Available	PASS
TC-15	FR-13	Wrong species: Place a cat with a dog-only foster	Alert shown; submission blocked	PASS
TC-16	FR-14	Valid foster placement: Place a dog with the foster	Animal With Foster; cage freed; foster Currently Fostering	PASS
TC-17	FR-13	Foster at capacity: Place a second animal with the same foster	Capacity alert; submission blocked	PASS
TC-18	FR-14	Complete a placement: Set placement status to Completed	Animal Ready for Release; foster Available	PASS
TC-19	FR-15	Treatment digest: Run the schedule with a treatment due tomorrow	Digest e-mail received by vet	PASS
TC-20	FR-15	Capacity alert: Fill cages above 90% and run the schedule	Administrator alert e-mail received	PASS
TC-21	FR-16	Release approval: Request release; approve as administrator	Approval recorded; approval e-mail sent	PASS
TC-22	FR-17	Foster carer view: Log in as a foster carer	Only own placements and animals visible	PASS
TC-23	FR-17	Volunteer view: Log in as a volunteer	Vet Notes, Dosage and Medication hidden	PASS

6.3 Debugging Deluge Scripts

Four techniques are used to debug Deluge scripts. First, info statements are placed after every fetch, update and function call so that values can be read in the Execute output or in the application logs under Operations > Logs. Second, temporary diagnostic alerts are placed in on-load and on-validate workflows to confirm the values of input fields before relying on them. Third, one-line probes, such as alert input.Cage in the validation workflow, confirm that a lookup field returns an ID before it is used in a fetch criterion. Fourth, the Execute button of the function editor is used to run updateCageOccupancy and refreshFosterAvailability with test arguments, and the result is checked in the cage and foster records.

Typical problems to watch for are a link name that differs from the label, a multi-select option whose text does not exactly match the drop-down values it is compared with, a workflow triggered on the wrong event, and a sendmail that needs a verified sender.

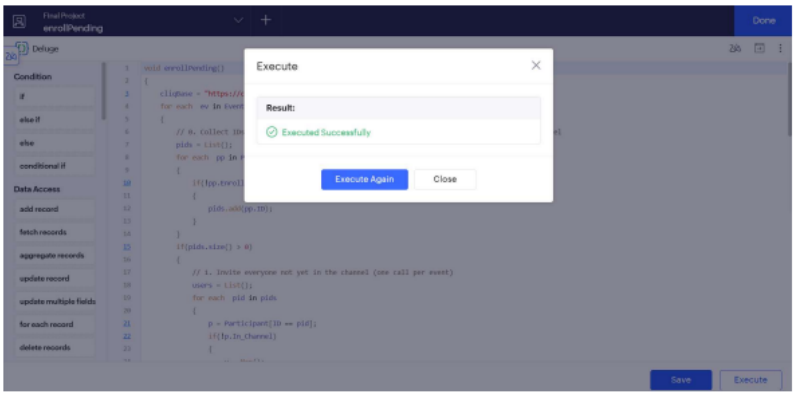


Figure 6.1: Running refreshFosterAvailability with Execute in the Deluge Function Editor

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Cliq channel creation returned {"code":"operation_failed"}	Channel name contained the "#" character	Replaced "#" with " E" in the generated name	Closed
BUG-02	Script error: argument type of subString did not match NUMBER	min() was used inside subString	Replaced with an explicit length check	Closed
BUG-03	Error "Variable expected is not defined"	A partial edit left a reference to a removed variable	Replaced the whole script with a consistent version	Closed

6.5 User Acceptance Testing

Acceptance testing is carried out by working through each role's tasks end to end, signing in with a separate account for each role on the web and on the Zoho Creator mobile app, exactly as a real user would. Table 6.3 records the problems noticed from the user's point of view and the action taken for each; related defects are listed in Table 6.2.

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
Kaaviya Sri PS (pskaaviya123@gmail.com)	Rescue Coordinator	At first saw every form and the dashboard	Permission set reassigned (BUG01)
Jhasmitha D (artemis.2126.2026@gmail.com)	Veterinarian	An announcement for one event appeared in every event's Cliq channel;	Guard clause in notifyEvent and event ID passed from the schedule (BUG03)

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

Zoho Creator provides Development, Stage and Production environments; changes are made in Development, published to Stage for verification as a versioned release, and then published to Production, where end users work [7]. The deployment procedure is: open Environments under Deploy, choose Publish to Stage, select the application and its changed components, enter a title and description; after verifying the Stage version, choose Publish to Production and select that version. Published versions cannot be rolled back, so the rollback plan is to publish a corrected version and to keep a .ds export of each released version as a backup. The application was published to Production on 31 October 2026 as version 1.2 (Figure 7.1).

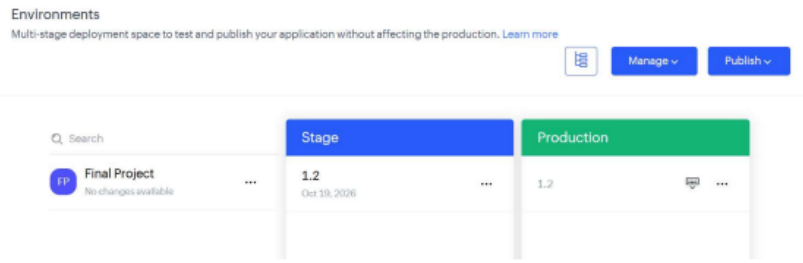


Figure 7.1: Environments – Publish to Stage and Production

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	All test cases executed in Development / Stage	✓
2	Sample / test data removed or isolated	✗
3	Roles, permission sets and users configured in Production	✓
4	Zoho CRM connection authorised in Production	✓
5	Schedules and workflows enabled	✓
6	Application published and URL verified	✓
7	Version / release notes recorded	✓

7.2 Performance Optimisation

Optimisation focused on keeping data correct and on limiting the amount of data each user loads. The measures are listed in Table 7.2; they are design choices and are described in terms of calls and records rather than timings.

Table 7.2: Performance Optimisation Measures

Optimisation Applied	Component	Before	After	Improvement
Recount occupancy by counting open assignments	updateCageOccupancy (DS-13)	Increment / decrement drifts after edits or deletions	Value recomputed from source records	Always-correct capacity; one reusable function
One digest e-mail instead of one per treatment	Treatment reminders (DS-16)	One e-mail per due treatment	One e-mail per day	Fewer e-mails and mail operations
Criteria on reports and dashboard elements	Foster reports and My Foster Dashboard	All records loaded then hidden	Only the carer's own records loaded	Smaller result sets per view
Shared functions for repeated logic	DS-13 to DS-15	Same code copied into several workflows	Single function called by all	Easier maintenance; fewer errors
Close old follow-ups on new treatment	Treatment Log (DS-07)	All past treatments stay "due"	Only the latest follow-up is open	Reminders list only genuinely due items

7.3 Security Features

Security works at four levels: who can open a component (permission sets), which records they can see (criteria), which fields they can see or change (field permissions and script-set fields) and how the application talks to other services (connections). The controls are listed in Table 7.3.

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Role-based permission sets	All forms, reports and pages	Users opening components outside their duties
Record-level criteria (Foster_Email == zoho.loginuserid)	My Fostered Animals, My Foster Dashboard, foster permission set	Foster carers seeing other carers' animals
Field permissions	Vet Notes, Dosage, Medication hidden from volunteers	Exposure of medical details
Read-only, script-set fields	Current Occupancy, Cage Status, Current Status, Follow-up Done	Tampering with capacity and status values
Validation workflows	Holding Assignment, Foster Placement, Injury Record, Treatment Log	Over-filled cages, over-allocated fosters, invalid dates
Approval process	Release of animals	Unauthorised release
Zoho connections (OAuth)	Zoho CRM integration, Creator REST API	Credential leakage; no hard-coded keys

Security Control	Where Applied	Threat Mitigated
Personal data flag	Foster e-mail, mobile, address	Identifies personal data for privacy controls

7.4 Governance and Administration

Logs and traceability. Every form action, schedule run, e-mail and integration call is recorded with a timestamp in Operations > Logs, which is used throughout testing. Figure 7.2 shows the log of schedule executions. Every record also keeps its Added User and Added Time.

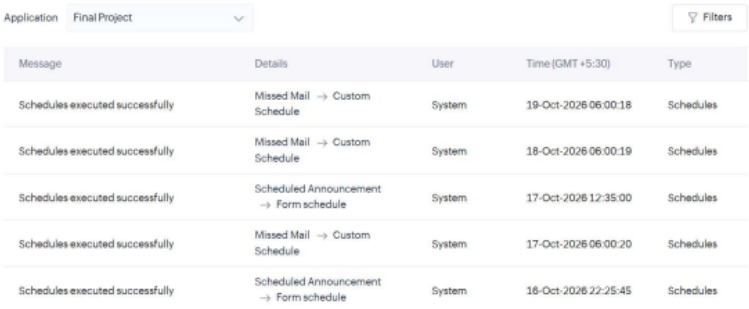


Figure 7.2: Schedule Execution Logs of the Application (Operations > Logs)

Administration. Users are managed centrally on the Creator account's Users page, where each user receives one role and one permission set per application. Only the administrator can build applications. The CRM connection is enabled only for this application. **Naming conventions:** fields use readable link names matching their labels, workflows are named after their purpose, and DS identifiers link scripts to this report. **Backup:** the application is exported as a .ds file after each milestone; the export is also the source of Appendix C.

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The following scenario demonstrates the application end to end. A volunteer rescues an injured dog and submits a Rescue Report from the mobile app with a photograph; because the condition is Critical, the status is set to In Medical Care and the veterinarian receives an alert e-mail (Figure 8.1). The veterinarian records a Severe fracture in the Injury Record and logs the first treatment with the next dose due in two days, which appears in the Treatment Calendar. The coordinator assigns the dog to a medical ward cage; the occupancy is recounted and, when a second animal fills the cage, a third assignment is refused with a "cage is full" message (Figure 8.2).

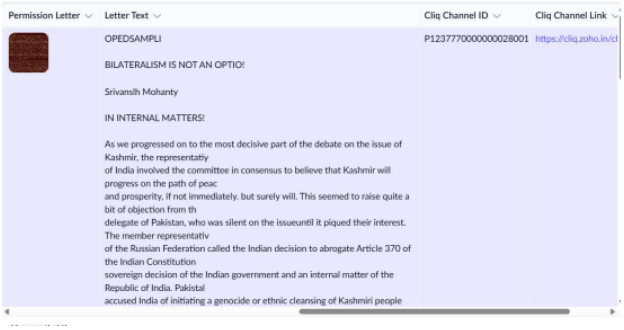


Figure 8.1: Scenario Step 1 – Critical Rescue Recorded and Vet Alerted

When the dog is stable, the coordinator creates a foster placement with a carer who accepts dogs and has capacity. The animal moves to With Foster, its cage is released and the carer's availability is updated (Figure 8.3). The carer sees the animal in My Fostered Animals and records a check-in; at the end of the placement the animal becomes Ready for Release, the administrator approves the release, and the cage and foster records are left correct. Throughout, the administrator's dashboard shows live occupancy, animals by status and treatments due.

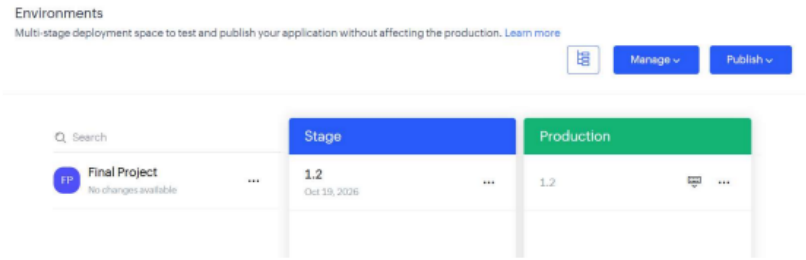


Figure 8.2: Scenario Step 2 – Cage Assignment and Full-Cage Alert

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Design a normalised data model and forms	Seven linked forms (Tables 4.1–4.8)	Sections 4.2, 4.3, 5.2
Automate triage, escalation, occupancy, foster validation and reminders	12 form workflows, 2 schedules and 3 functions	Sections 5.5, 5.6; TC-02 to TC-20
Prevent over-filled cages and over-allocated fosters	Validation workflows DS-08 and DS-10	TC-11, TC-12, TC-15, TC-17
Role-based dashboards and reports	Permission sets, criteria, Shelter and Foster dashboards	Sections 5.3, 5.4, 5.8; TC-22, TC-23
Integrate with Zoho CRM	Integration field and CRM contact creation	Section 5.7
Deploy securely with governance controls	Environments, permissions, logs	Chapter 7

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Rescue records	Notebook or chat messages	Structured Rescue Report with photo and condition	Complete, searchable history
Injuries and treatment	Loose paper and memory	Injury Record and Treatment Log with due dates	Full medical history per animal
Follow-up reminders	None	Daily digest and instant alerts	Fewer missed treatments
Cage capacity	Remembered by staff	Validated and recounted automatically	No over-filled cages
Foster tracking	Phone calls	Validated placements with availability	Clear view of who holds which animal
Access control	Everyone sees everything	Roles, field permissions, record criteria	Sensitive data protected

Parameter	Existing System	Proposed Application	Improvement
Management overview	Manual counting	Live dashboard and charts	Real-time visibility

8.4 Limitations of the Low-Code Approach

Several real limits of the platform are relevant to this application. **UI customisation:** forms and reports follow Creator's layout, so a custom visual such as a cage map would need a widget. **Scripting constraints:** Deluge has restrictions, such as limits on what can be stored in collections and on how criteria are written in fetch statements, which require workarounds. **Platform limits:** schedules, approvals, integrations and API calls are subject to plan limits and rate limits. **Report criteria:** some report types, such as pivot charts, do not support every criterion, so a user-specific pivot view may not be possible. **Vendor lock-in and cost:** the application runs only on Zoho Creator, exports only as a .ds file that can be re-imported into Creator, and premium features require a paid plan after the trial. Each limitation can be handled with a workaround, and none prevents the core requirements from being met.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

No-code parts are built purely through configuration: the forms and field properties, lookups, the list, kanban, calendar and pivot reports, dashboard pages, permission sets and roles, the approval routing and the connections. Low-code parts need Deluge: validations, status escalation, cage recounting, foster validation, reminders and the CRM call. Pro-code was not needed in this project; the nearest step would be a JavaScript widget for a cage map or an external service for heavy processing.

This distribution, in which most of the application is configured and the business rules are scripted, reflects where each approach is strongest. The project would move further towards pro-code if it needed a custom public-facing interface, bulk analysis of large histories or integration with veterinary clinic systems; at that point an external service such as Zoho Catalyst or a separately hosted application would be more appropriate than more Deluge.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

This project built the Pet Rescue, Rehabilitation and Shelter Management System, a Zoho Creator application that follows an animal from rescue to a cage or a foster home. Staff record each rescue with its condition on arrival, log injuries and treatments, and let the application escalate serious cases, remind the vet of due treatments and keep cage occupancy correct. Foster placements are validated against capacity and species, release is approved by the administrator, and each role works from its own dashboards and reports with sensitive details protected.

The project used the major capabilities of the platform covered in the course: data modelling, forms, reports, pages, Deluge, form and scheduled workflows, approvals, integration with Zoho CRM, role-based access and environments. It showed how quickly a complete operational application can be built with low-code tools and where those tools reach their limits.

9.2 Future Enhancements

- A Blueprint for the animal's care pathway (Intake, Medical, Rehab, Ready, Released) with mandatory fields at each stage.
- A JavaScript widget showing a visual cage map with colour-coded occupancy.
- A customer portal for foster carers, so that external carers have a simpler, separate login.
- Inventory and donation modules for food, medicine and supplies, with low-stock alerts.
- Automatic vaccination schedules by species and age, generated as treatment reminders.
- Integration with the adoption application being developed separately, so that animals marked Ready for Release can be listed for adoption.
- Zia-based analysis of intake and recovery data to predict length of stay and capacity pressure.

9.3 Learning Outcomes and Career Relevance

The project developed skills across the low-code spectrum: modelling data with forms and lookups, writing and debugging Deluge, designing role-based access with record-level criteria, integrating with Zoho CRM, reading platform documentation and managing releases through environments. It also showed the value of recounting rather than incrementing derived values, of validating data before it is saved, and of keeping a defect log.

These skills map directly to roles such as citizen developer within an organisation, low-code developer, Zoho Creator or Zoho One solutions consultant and automation engineer. The experience of extending the platform with reusable functions and integrations is also a step towards professional software development, which is where the course's no-code → low-code → pro-code spectrum leads.

REFERENCES

[1]Zoho Corporation, “Zoho Creator Help.” [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 9 November 2026).

[2]A. C. Bock and U. Frank, “Low-code platform,” Business & Information Systems Engineering, vol. 63, no. 6, pp. 733–740, 2021, doi: 10.1007/s12599-021-00726-8.

[3]D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi, and M. Wimmer, “Low-code development and model-driven engineering: Two sides of the same coin?,” Software and Systems Modeling, vol. 21, pp. 437–446, 2022, doi: 10.1007/s10270-021-00970-2.

[4]Zoho Corporation, “Deluge Script Reference Guide.” [Online]. Available: https://www.zoho.com/deluge/ (Accessed: 9 November 2026).

[5]A. Sahay, A. Indamutsa, D. Di Ruscio, and A. Pierantonio, “Supporting the understanding and comparison of low-code development platforms,” in Proc. 46th Euromicro Conf. Software Engineering and Advanced Applications (SEAA), 2020, pp. 171–178.

[6]M. A. A. Alamin, S. Malakar, G. Uddin, S. Afroz, T. B. Haider, and A. Iqbal, “An empirical study of developer discussions on low-code software development challenges,” arXiv:2103.11429, 2021.

[7]Zoho Corporation, “Publishing applications to stage and production environments | Zoho Creator Help.” [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/environments/articles/environments-locking-mechanism (Accessed: 9 November 2026).

[8]Zoho Corporation, “Zoho CRM Developer Documentation.” [Online]. Available: https://www.zoho.com/crm/developer/docs/ (Accessed: 9 November 2026).

[9]Zoho Corporation, “Zoho Creator REST API v2.1.” [Online]. Available: https://www.zoho.com/creator/help/api/v2.1/ (Accessed: 9 November 2026).

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	Rescue, Rehabilitation, Shelter, Dashboard and My Fostered Animals sections	5.1	Fig. 5.1
II	Forms with ≥ 8 distinct field types	Y	17 field types incl. Single Line, Multi Line, Number, Decimal, Date, Date-Time, Email, Phone, Name, Address, Image, Decision Box, Drop Down, Radio, Multi Select, Lookup, Auto Number	5.2	Tables 4.2–4.8
II	Lookup field (relationship between forms)	Y	Injury, Treatment, Holding, Placement → Rescue Report; Holding → Cage; Placement → Foster	4.2, 5.2	Table 4.1
II	Subform (parent–child data)	N	Not required by the data model	—	—
II	Field validations / layout rules	Y	On Validate workflows; defaults; read-only script-set fields	5.2, 5.5	Code 5.2
II	List report with filter, grouping, custom action	Partial	Eight list reports with criteria and grouping; no custom action	5.3	Table 5.1
II	Kanban / Calendar / Timeline / Map report	Y	Animal Status Board; Treatment Calendar	5.3	Fig. 5.4
II	Pivot table / chart report	Y	Occupancy Chart	5.3, 5.9	Table 5.1
II	Page / dashboard built with UI Builder	Y	Shelter Dashboard; My Foster Dashboard	5.4	Fig. 5.5
II	Deluge – form events (validate / success / field actions)	Y	DS-01 to DS-12	5.5	Code 5.2, C.1–C.12
II	Deluge – functions, fetch, loops, collections	Y	updateCageOccupancy, refreshFosterAvailability, closeCageAssignments	5.5	Code 5.1, C.13–C.15
II	Email / in-app notifications	Y	Critical, severe-injury, declining, concern, digest and capacity e-mails	5.5, 5.6	DS-02, DS-16, DS-17
II	Form workflow	Y	12 form workflows	5.6	Table 5.3
II	Scheduled workflow	Y	2 schedules (treatment reminders, capacity alert)	5.6	Table 5.3
II	Approval process	Y	Release approval on Rescue Report	5.6	DS-18
II	Blueprint	N	Listed as a future enhancement	9.2	—
II	Batch / report workflow	N	Not required	5.6	—
II	Testing & debugging evidence	Y	23 test cases; defect log	6	Tables 6.1, 6.2
III	Integration with a Zoho service	Y	Zoho CRM contacts	5.7	Table 5.4
III	Connection / REST API call	Y	Zoho CRM connection; zoho.crm.createRecord	5.7	Code 5.3
III	Webhook or Creator REST API	Y	Creator REST API v2.1 GET from Postman	5.7	Table 5.4

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
III	Users, roles and permission profiles	Y	Six roles and permission sets	5.8	Fig. 5.6, Table 4.9
III	Record-level access criteria / customer portal	Y	Foster_Email == zoho.loginuserid criteria	5.8	Table 5.1
III	Zia / AI feature	Partial	AI-assisted development declared; no Zia field used	5.9	—
III	Analytics (charts / Zoho Analytics)	Y	Dashboard charts and KPI panels; pivot chart	5.9	Fig. 5.5
IV	Performance optimisation applied & measured	Partial	Design measures described; not timed	7.2	Table 7.2
IV	Deployment across environments	Y	Development → Stage → Production	7.1	Fig. 7.1
IV	Security controls (encryption, OAuth, permissions)	Y	Permissions, field permissions, criteria, connections	7.3	Table 7.3
IV	Governance (audit trail, administration)	Y	Operations logs; user administration	7.4	Fig. 7.2
V	Pro-code extension (Catalyst / widgets / custom functions)	Y	Three reusable Deluge custom functions	5.10	C.13–C.15
V	Low-code limitations & spectrum analysis	Y	Limitations and spectrum analysis	8.4, 8.5	—
—	Mobile app / portal access	Y	Zoho Creator mobile app for caretakers, fosters and volunteers	5.11	—

Total features fully implemented: 26 / 32 (partially implemented: 3; not implemented: 3).

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Assessed Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	3 – High	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	3 – High	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9	3 – High	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	2 – Medium	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	2 – Medium	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 Rescue Report – On Load (DS-01)

```
// Script ID : DS-01
// Location  : Rescue Report -> Load of the form (create)
// Purpose   : Default status and rescue time
input.Current_Status = "In Holding";
input.Rescue_Date_Time = zoho.currenttime;
```

C.2 Rescue Report – Triage and Critical Alert (DS-02)

```
// Script ID : DS-02
// Location  : Rescue Report -> Successful form submission (created)
// Purpose   : Set triage status; alert the vet for critical animals
if(input.Condition_on_Arrival == "Critical" || input.Condition_on_Arrival == "Serious")
{
    input.Current_Status = "In Medical Care";
}
if(input.Condition_on_Arrival == "Critical")
{
    sendmail
    [
        from :zoho.adminuserid
        to :"vet@yourshelter.org"
        subject :"CRITICAL rescue admitted: " + input.Animal_ID
        message :"A critical-condition " + input.Species + " was rescued at " +
input.Rescue_Location + ". Please attend immediately."
    ]
}
```

C.3 Rescue Report – Release Housekeeping (DS-03)

```
// Script ID : DS-03
// Location  : Rescue Report -> Successful form submission (edited)
// Purpose   : Free cages when the animal is released or deceased
if(input.Current_Status == "Released" || input.Current_Status == "Deceased")
{
    thisapp.closeCageAssignments(input.ID);
    input.Current_Cage = null;
}
```

C.4 Injury Record – Validate (DS-04)

```
// Script ID : DS-04
// Location  : Injury Record -> Validate on submit (created)
// Purpose   : Reject future injury dates
if(input.Injury_Date_Noted > zoho.currentdate)
{
    alert "Injury date cannot be in the future.";
    cancel submit;
}
```

C.5 Injury Record – Escalate Severe Injury (DS-05)

```
// Script ID : DS-05
// Location  : Injury Record -> Successful form submission (created)
// Purpose   : Escalate severe injuries and notify the vet
if(input.Severity == "Severe")
{
```

```
    animalRec = Rescue_Report[ID == input.Animal];
    if(animalRec.Current_Status != "With Foster")
    {
        animalRec.Current_Status = "In Medical Care";
    }
    sendmail
    [
        from :zoho.adminuserid
        to :"vet@yourshelter.org"
        subject :"Severe injury logged: " + animalRec.Animal_ID
        message :"Type: " + input.Injury_Type + ", Body part: " +
input.Body_Part_Affected + ". Please review."
    ]
}
```

C.6 Treatment Log – Validate (DS-06)

```
// Script ID : DS-06
// Location  : Treatment Log -> Validate on submit (created)
// Purpose   : Check the next-treatment date
if(input.Next_Treatment_Due != null && input.Next_Treatment_Due < input.Treatment_Date)
{
    alert "Next treatment due cannot be before the treatment date.";
    cancel submit;
}
```

C.7 Treatment Log – On Success (DS-07)

```
// Script ID : DS-07
// Location  : Treatment Log -> Successful form submission (created)
// Purpose   : Declining alert; close earlier follow-ups
animalRec = Rescue_Report[ID == input.Animal];
if(input.Progress == "Declining")
{
    if(animalRec.Current_Status != "With Foster")
    {
        animalRec.Current_Status = "In Medical Care";
    }
    sendmail
    [
        from :zoho.adminuserid
        to :"vet@yourshelter.org"
        subject :"Animal declining: " + animalRec.Animal_ID
        message :"Latest treatment log marks progress as Declining. Please review the
treatment plan."
    ]
}
oldLogs = Treatment_Log[Animal == input.Animal && ID != input.ID && Follow_up_Done ==
false];
for each o in oldLogs
{
    o.Follow_up_Done = true;
}
```

C.8 Holding Assignment – Validate (DS-08)

```
// Script ID : DS-08
// Location  : Holding Assignment -> Validate on submit (created)
// Purpose   : Block full or unavailable cages
cageRec = Cage_Master[ID == input.Cage];
if(cageRec.Cage_Status == "Out of Service" || cageRec.Cage_Status == "Under Cleaning")
{
    alert "This cage is " + cageRec.Cage_Status + ". Choose another cage.";
    cancel submit;
}
```

```
if(cageRec.Current_Occupancy >= cageRec.Capacity)
{
    alert "Cage " + cageRec.Cage_ID + " is full. Choose another cage.";
    cancel submit;
}
```

C.9 Holding Assignment – On Success (DS-09)

```
// Script ID : DS-09
// Location  : Holding Assignment -> Successful form submission (created or edited)
// Purpose   : Close old assignments, recount cages, sync the animal
otherOpen = Holding_Assignment[Animal == input.Animal && Moved_Out_Date == null && ID !=
input.ID];
for each a in otherOpen
{
    oldCage = a.Cage;
    a.Moved_Out_Date = zoho.currentdate;
    thisapp.updateCageOccupancy(oldCage);
}
thisapp.updateCageOccupancy(input.Cage);
animalRec = Rescue_Report[ID == input.Animal];
if(input.Moved_Out_Date == null)
{
    animalRec.Current_Cage = input.Cage;
    if(animalRec.Current_Status == "With Foster" || animalRec.Current_Status == "Ready
for Release")
    {
        animalRec.Current_Status = "In Holding";
    }
}
```

C.10 Foster Placement – Validate (DS-10)

```
// Script ID : DS-10
// Location  : Foster Placement -> Validate on submit (created)
// Purpose   : Availability, capacity and species checks
fosterRec = Foster_Master[ID == input.Foster];
animalRec = Rescue_Report[ID == input.Animal];
if(fosterRec.Availability == "On Break")
{
    alert "This foster carer is currently on break.";
    cancel submit;
}
activeCount = Foster_Placement[Foster == input.Foster && Placement_Status ==
"Active"].count();
if(activeCount >= fosterRec.Max_Animals)
{
    alert "This foster is already at capacity (" + fosterRec.Max_Animals + " animals).";
    cancel submit;
}
if(!fosterRec.Species_Accepted.contains(animalRec.Species))
{
    alert "This foster does not accept " + animalRec.Species + ".";
    cancel submit;
}
if(input.Expected_End_Date != null && input.Expected_End_Date <
input.Placement_Start_Date)
{
    alert "Expected end date cannot be before the start date.";
    cancel submit;
}
```

C.11 Foster Placement – On Success, created (DS-11)

```
// Script ID : DS-11
```

```
// Location  : Foster Placement -> Successful form submission (created)
// Purpose   : Move the animal to the foster; free the cage
animalRec = Rescue_Report[ID == input.Animal];
animalRec.Current_Status = "With Foster";
animalRec.Current_Cage = null;
thisapp.closeCageAssignments(input.Animal);
thisapp.refreshFosterAvailability(input.Foster);
fosterRec = Foster_Master[ID == input.Foster];
input.Foster_Email = fosterRec.Email;
```

C.12 Foster Placement – On Success, edited (DS-12)

```
// Script ID : DS-12
// Location  : Foster Placement -> Successful form submission (edited)
// Purpose   : Completion, early return and concern alert
animalRec = Rescue_Report[ID == input.Animal];
if(input.Placement_Status == "Completed")
{
    animalRec.Current_Status = "Ready for Release";
}
else if(input.Placement_Status == "Returned Early")
{
    animalRec.Current_Status = "In Holding";
}
thisapp.refreshFosterAvailability(input.Foster);
if(input.Animal_Condition_at_Check_in == "Concern")
{
    sendmail
    [
        from :zoho.adminuserid
        to :"vet@yourshelter.org"
        subject :"Foster check-in concern: " + animalRec.Animal_ID
        message :"The foster reported a concern at check-in. Please follow up."
    ]
}
```

C.13 Function – updateCageOccupancy (DS-13)

```
// Script ID : DS-13
// Location  : Function updateCageOccupancy(int cageId)
// Purpose   : Recount open assignments and set cage status
cageRec = Cage_Master[ID == cageId];
occ = Holding_Assignment[Cage == cageId && Moved_Out_Date == null].count();
cageRec.Current_Occupancy = occ;
if(cageRec.Cage_Status != "Out of Service" && cageRec.Cage_Status != "Under Cleaning")
{
    if(occ >= cageRec.Capacity)
    {
        cageRec.Cage_Status = "Full";
    }
    else
    {
        cageRec.Cage_Status = "Available";
    }
}
```

C.14 Function – refreshFosterAvailability (DS-14)

```
// Script ID : DS-14
// Location  : Function refreshFosterAvailability(int fosterId)
// Purpose   : Recompute foster availability
fosterRec = Foster_Master[ID == fosterId];
activeCount = Foster_Placement[Foster == fosterId && Placement_Status ==
"Active"].count();
if(fosterRec.Availability != "On Break")
```

```
{
    if(activeCount >= fosterRec.Max_Animals)
    {
        fosterRec.Availability = "Currently Fostering";
    }
    else
    {
        fosterRec.Availability = "Available";
    }
}
```

C.15 Function – closeCageAssignments (DS-15)

```
// Script ID : DS-15
// Location  : Function closeCageAssignments(int animalId)
// Purpose   : Close every open cage assignment of an animal
openList = Holding_Assignment[Animal == animalId && Moved_Out_Date == null];
for each a in openList
{
    cageId = a.Cage;
    a.Moved_Out_Date = zoho.currentdate;
    thisapp.updateCageOccupancy(cageId);
}
```

C.16 Schedule – Treatment Reminders (DS-16)

```
// Script ID : DS-16
// Location  : Schedule: Treatment Reminders (daily, 08:00 IST)
// Purpose   : E-mail a digest of treatments due today or tomorrow
dueList = Treatment_Log[Next_Treatment_Due != null && Next_Treatment_Due <=
zoho.currentdate.addDay(1) && Follow_up_Done == false];
msg = "";
for each t in dueList
{
    msg = msg + t.Animal.Animal_Name + " (" + t.Animal.Animal_ID + "): " +
t.Treatment_Type + " due " + t.Next_Treatment_Due + "<br>";
}
if(dueList.count() > 0)
{
    sendmail
    [
        from :zoho.adminuserid
        to :"vet@yourshelter.org"
        subject :"Treatments due: " + dueList.count() + " animal(s)"
        message :msg
    ]
}
```

C.17 Schedule – Capacity Alert (DS-17)

```
// Script ID : DS-17
// Location  : Schedule: Capacity Alert (daily, 09:00 IST)
// Purpose   : Alert the administrator at 90% occupancy
activeCages = Cage_Master[Cage_Status != "Out of Service"];
totalCap = ifnull(activeCages.sum(Capacity),0);
totalOcc = ifnull(activeCages.sum(Current_Occupancy),0);
if(totalCap > 0)
{
    pct = (totalOcc * 100) / totalCap;
    if(pct >= 90)
    {
        sendmail
        [
            from :zoho.adminuserid
            to :"admin@yourshelter.org"
```

```
            subject :"Shelter near capacity: " + pct.round(0) + "% full"
            message :totalOcc + " of " + totalCap + " spaces are occupied. Consider
fostering or transfers."
        ]
    }
}
```

C.18 Approval – Release Approval (DS-18)

```
// Script ID : DS-18
// Location  : Approval process on Rescue Report -> On approve / On reject
// Purpose   : Notify on approval; revert status on rejection
// On approve
rec = Rescue_Report[ID == input.ID];
sendmail
[
    from :zoho.adminuserid
    to :"coordinator@yourshelter.org"
    subject :"Release approved: " + rec.Animal_ID
    message :"The release of " + rec.Animal_ID + " has been approved by the
administrator."
]
// On reject
rec = Rescue_Report[ID == input.ID];
rec.Current_Status = "In Rehab";
```

C.19 Foster Master – Create CRM Contact

```
// Location  : Foster Master -> Successful form submission (created)
// Purpose   : Create the matching Zoho CRM contact
contactMap = Map();
contactMap.put("First_Name", input.Foster_Name.first_name);
contactMap.put("Last_Name", input.Foster_Name.last_name);
contactMap.put("Email", input.Email);
contactMap.put("Mobile", input.Mobile);
resp = zoho.crm.createRecord("Contacts", contactMap);
info resp;
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

1. Accept the invitation e-mail from Zoho Creator and sign in with the same e-mail address.
2. On the web, open Zoho Creator, select the correct account if the account switcher shows more than one, then open the Pet Rescue application.
3. On a phone, install the Zoho Creator app, sign in with the same account and open the application. Allow camera access when asked.
4. Foster carers must sign in with the e-mail address registered in Foster Master, because their records are matched by e-mail.

D.2 Instructions for Staff (Administrator, Veterinarian, Coordinator, Caretaker)

- **Record a rescue:** open Rescue > Rescue Report, enter species, location, rescue type, condition and behaviour on arrival, add a photo and submit. Critical animals trigger an e-mail to the vet.
- **Record an injury:** open Rescue > Injury Record, select the animal, enter type, body part and severity, and submit. A severe injury moves the animal to In Medical Care.
- **Log a treatment:** open Rehabilitation > Treatment Log, select the animal, enter the treatment and progress, and set Next Treatment Due. Due treatments appear in the Treatment Calendar and the daily e-mail digest.
- **Assign a cage:** open Shelter > Holding Assignment, select the animal and a cage. If the cage is full or unavailable, a message appears; choose another cage. Use Moved Out Date when the animal leaves the cage.
- **Register a foster and place an animal:** add the carer in Foster Master, then open Foster Placement, select the animal and the carer, and submit. The system checks capacity and species.
- **Monitor:** use the Shelter Dashboard for occupancy, animals by status and treatments due; the administrator approves releases under Approvals.

D.3 Instructions for Foster Carers and Volunteers

- **Foster carers:** open My Fostered Animals to see the animals in your care and their care instructions. Record a check-in with the date and the animal's condition; choose Concern if the animal seems unwell so that the vet is alerted.
- **Foster carers:** when a placement ends, tell the coordinator so that the status can be changed to Completed or Returned Early.
- **Volunteers:** open Rescue > Rescue Report to report a rescue with a photograph. You can also view the cage list and the holding register to see where space is available.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name		Register No.	
Project Title		Date	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name		Register No.	
Department / Section		Date of Viva	
Application URL Verified	Yes / No	Demo Duration	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER

Name: _____

EXTERNAL EXAMINER

Name: _____